

Creating a Sequence and Taking the variable to be predicted

```
In [ ]: import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler
import joblib
import tensorflow as tf

SEQ_LEN = 60          # sequence length (use 30/60/90 as experiments)
FEATURES = ['Close']  # change to ['Open', 'High', 'Low', 'Close', 'Volume']
CSV_PATH = "gold_futures_full_history.csv"
```

Preprocessing of the Dataset

```
In [ ]: df = pd.read_csv(CSV_PATH)
if 'Date' in df.columns:
    df['Date'] = pd.to_datetime(df['Date'])
    df = df.sort_values('Date').reset_index(drop=True)
df = df.drop_duplicates().reset_index(drop=True)
print("Loaded rows:", len(df))
```

Loaded rows: 1261

```
In [ ]: print(df.columns)
print(df[FEATURES].isna().sum())
# Drop rows with any NaN in features (keeps things safe)
df = df.dropna(subset=FEATURES).reset_index(drop=True)
```

```
Index(['Date', 'Open', 'High', 'Low', 'Close', 'Adj Close', 'Volume'], dtype='object')
Close    3
dtype: int64
```

```
In [ ]: # For univariate (recommended): keep only Close
data = df[FEATURES].astype('float32').values # shape (N, num_features)
print("data shape:", data.shape)
```

data shape: (1258, 1)

Min-Max Scaling

```
In [ ]: scaler = MinMaxScaler(feature_range=(0,1))
data_scaled = scaler.fit_transform(data) # (N, num_features)
joblib.dump(scaler, "scaler_minmax.save") # save to invert later
```

Out[]: ['scaler_minmax.save']

Creating a Sequence Length

```
In [ ]: def create_sequences(arr, seq_len=SEQ_LEN):
    X, y = [], []
    for i in range(seq_len, len(arr)):
        X.append(arr[i-seq_len:i]) # (seq_len, num_features)
        y.append(arr[i])           # next step (num_features)
    X = np.array(X)
    y = np.array(y)
    return X, y
```

```
X, y = create_sequences(data_scaled, seq_len=SEQ_LEN)
print("X shape:", X.shape, "y shape:", y.shape)
# If univariate, y shape = (samples,1). If you want scalar y: y = y.reshape(-1,1)
```

X shape: (1198, 60, 1) y shape: (1198, 1)

```
In [ ]: total = len(X)
train_end = int(total * 0.70)
val_end = int(total * 0.85)

X_train, y_train = X[:train_end], y[:train_end]
X_val, y_val = X[train_end:val_end], y[train_end:val_end]
X_test, y_test = X[val_end:], y[val_end:]

print("train/val/test:", X_train.shape, X_val.shape, X_test.shape)
```

train/val/test: (838, 60, 1) (180, 60, 1) (180, 60, 1)

```
In [ ]: BATCH = 32
train_ds = tf.data.Dataset.from_tensor_slices((X_train, y_train)).batch(BATCH)
val_ds = tf.data.Dataset.from_tensor_slices((X_val, y_val)).batch(BATCH)
test_ds = tf.data.Dataset.from_tensor_slices((X_test, y_test)).batch(BATCH)
```

```
In [ ]: print("Shapes (X_train,y_train):", X_train.shape, y_train.shape)
print("Any NaNs in X_train:", np.isnan(X_train).sum(), "y_train:", np.isnan(y_train).sum())
# If multivariate and you want target as scalar Close only:
# y = y[:, FEATURES.index('Close')].reshape(-1,1)
```

Shapes (X_train,y_train): (838, 60, 1) (838, 1)

Any NaNs in X_train: 0 y_train: 0

Building the GRU Model

```
In [ ]: import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import GRU, Dense, Dropout
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping, ReduceLRonPlateau

num_features = X_train.shape[-1] # 1 for univariate, >1 for multivariate

model = Sequential([
    GRU(128, return_sequences=True, input_shape=(SEQ_LEN, num_features)),
    Dropout(0.2),

    GRU(64),
    Dropout(0.2),

    Dense(num_features) # Predict same shape as y (1 for univariate)
])

model.compile(
    loss='mse',
    optimizer=Adam(0.001),
    metrics=['mae']
)

model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(**kwargs)
```

Model: "sequential_4"

Layer (type)	Output Shape	
gru_8 (GRU)	(None, 60, 128)	
dropout_8 (Dropout)	(None, 60, 128)	
gru_9 (GRU)	(None, 64)	
dropout_9 (Dropout)	(None, 64)	
dense_4 (Dense)	(None, 1)	

Total params: 87,617 (342.25 KB)

Trainable params: 87,617 (342.25 KB)

Non-trainable params: 0 (0.00 B)

Training the GRU Model

```
In [ ]: early_stop = EarlyStopping(monitor='val_loss', patience=6, restore_best_weights=True)
        lr_sched = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=3)

        history = model.fit(
            train_ds,
            validation_data=val_ds,
            epochs=80,
            callbacks=[early_stop, lr_sched],
            verbose=1
        )
```

Epoch 1/80
27/27  10s 122ms/step - loss: 0.0012 - mae: 0.0280 - val_loss: 6.9412e-04 - val_mae: 0.0228 - learning_rate: 0.0010

Epoch 2/80
27/27  3s 128ms/step - loss: 4.4548e-04 - mae: 0.0175 - val_loss: 1.9336e-04 - val_mae: 0.0113 - learning_rate: 0.0010

Epoch 3/80
27/27  4s 128ms/step - loss: 4.3789e-04 - mae: 0.0171 - val_loss: 1.4853e-04 - val_mae: 0.0095 - learning_rate: 0.0010

Epoch 4/80
27/27  3s 104ms/step - loss: 1.4650e-04 - mae: 0.0096 - val_loss: 1.3075e-04 - val_mae: 0.0093 - learning_rate: 0.0010

Epoch 5/80
27/27  5s 105ms/step - loss: 1.8864e-04 - mae: 0.0111 - val_loss: 5.9010e-04 - val_mae: 0.0219 - learning_rate: 0.0010

Epoch 6/80
27/27  5s 103ms/step - loss: 1.3322e-04 - mae: 0.0093 - val_loss: 1.3937e-04 - val_mae: 0.0096 - learning_rate: 5.0000e-04

Epoch 7/80
27/27  3s 104ms/step - loss: 1.6520e-04 - mae: 0.0099 - val_loss: 1.3225e-04 - val_mae: 0.0091 - learning_rate: 5.0000e-04

Epoch 8/80
27/27  3s 102ms/step - loss: 1.8625e-04 - mae: 0.0108 - val_loss: 1.2574e-04 - val_mae: 0.0091 - learning_rate: 5.0000e-04

Epoch 9/80
27/27  4s 153ms/step - loss: 1.0043e-04 - mae: 0.0078 - val_loss: 1.2805e-04 - val_mae: 0.0092 - learning_rate: 2.5000e-04

Epoch 10/80
27/27  3s 103ms/step - loss: 9.4994e-05 - mae: 0.0074 - val_loss: 1.2152e-04 - val_mae: 0.0090 - learning_rate: 2.5000e-04

Epoch 11/80
27/27  3s 104ms/step - loss: 1.0740e-04 - mae: 0.0080 - val_loss: 1.1840e-04 - val_mae: 0.0087 - learning_rate: 2.5000e-04

Epoch 12/80
27/27  7s 180ms/step - loss: 1.0533e-04 - mae: 0.0078 - val_loss: 1.1969e-04 - val_mae: 0.0086 - learning_rate: 1.2500e-04

Epoch 13/80
27/27  3s 110ms/step - loss: 1.0528e-04 - mae: 0.0078 - val_loss: 1.8666e-04 - val_mae: 0.0113 - learning_rate: 1.2500e-04

Epoch 14/80
27/27  3s 105ms/step - loss: 1.2116e-04 - mae: 0.0086 - val_loss: 1.1742e-04 - val_mae: 0.0088 - learning_rate: 1.2500e-04

Epoch 15/80
27/27  7s 175ms/step - loss: 1.1350e-04 - mae: 0.0083 - val_loss: 1.4057e-04 - val_mae: 0.0097 - learning_rate: 6.2500e-05

Epoch 16/80
27/27  3s 108ms/step - loss: 9.3716e-05 - mae: 0.0073 - val_loss: 1.4658e-04 - val_mae: 0.0099 - learning_rate: 6.2500e-05

Epoch 17/80
27/27  3s 104ms/step - loss: 9.9371e-05 - mae: 0.0075 - val_loss: 2.0054e-04 - val_mae: 0.0118 - learning_rate: 6.2500e-05

Epoch 18/80
27/27  6s 123ms/step - loss: 1.0616e-04 - mae: 0.0077 - val_loss: 1.6594e-04 - val_mae: 0.0107 - learning_rate: 3.1250e-05

Epoch 19/80
27/27  5s 104ms/step - loss: 9.3303e-05 - mae: 0.0071 - val_loss: 1.2085e-04 - val_mae: 0.0089 - learning_rate: 3.1250e-05

Epoch 20/80
27/27  3s 104ms/step - loss: 9.0854e-05 - mae: 0.0072 - val_loss: 1.2231e-04 - val_mae: 0.0090 - learning_rate: 3.1250e-05

Prediction of the Model

```
In [ ]: pred_scaled = model.predict(test_ds)
        pred_scaled = pred_scaled.reshape(-1, num_features)
```

WARNING:tensorflow:5 out of the last 17 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x7c80b18b7740> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

6/6 ————— 1s 91ms/step

```
In [ ]: pred = scaler.inverse_transform(pred_scaled)
        y_test_orig = scaler.inverse_transform(y_test)
```

Metrics for evaluating performance of the Model

```
In [ ]: from sklearn.metrics import mean_squared_error, mean_absolute_error

        rmse = np.sqrt(mean_squared_error(y_test_orig, pred))
        mae = mean_absolute_error(y_test_orig, pred)
        mape = np.mean(np.abs((y_test_orig - pred) / y_test_orig)) * 100

        print("GRU RMSE :", rmse)
        print("GRU MAE :", mae)
        print("GRU MAPE :", mape)
```

GRU RMSE : 64.80756143793138

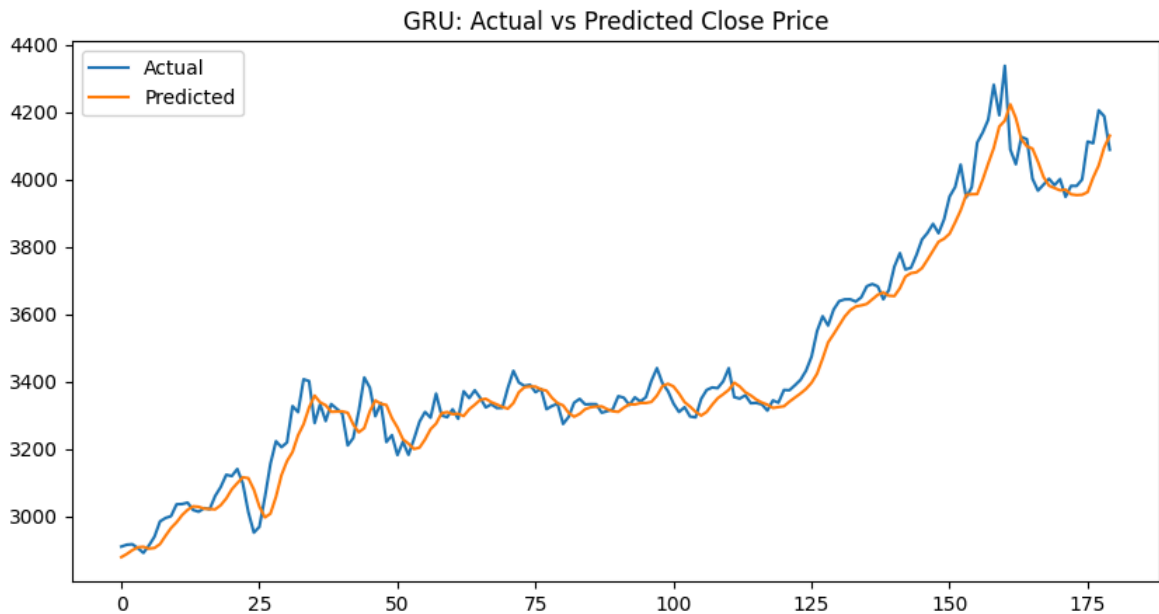
GRU MAE : 49.33497619628906

GRU MAPE : 1.4026474

Graph-Actual vs Predicted

```
In [ ]: # Step 14
        import matplotlib.pyplot as plt

        plt.figure(figsize=(10,5))
        plt.plot(y_test_orig, label="Actual")
        plt.plot(pred, label="Predicted")
        plt.title("GRU: Actual vs Predicted Close Price")
        plt.legend()
        plt.show()
```



Training and Validation Loss Graph

```
In [ ]: # Step 15
plt.figure(figsize=(10,4))
plt.plot(history.history['loss'], label="train_loss")
plt.plot(history.history['val_loss'], label="val_loss")
plt.title("Training & Validation Loss")
plt.legend()
plt.show()
```



1.Huge improvement: GRU errors (RMSE 64, MAE 49, MAPE 1.4%) are far lower than the RNN model.

2.High forecast accuracy: With only 1–2% average deviation, the GRU model is reliable for planning, budgeting, and demand forecasting.

3.Better risk management: Low error variance means fewer unexpected spikes → more stable decisions.

4.Operational benefits: You can confidently use this model for inventory planning, cost optimization, and resource allocation.

5.Conclusion: GRU provides high-precision predictions and is suitable for real-world business deployment.